

Velox Technical Foundation Pack

1. Purpose of This Pack

The Velox Technical Foundation Pack is the starting system for turning Velox from an idea into a working AI-powered creator intelligence platform.

Velox is not just a dashboard. It is an AI creator intelligence system that discovers emerging creators, analyzes their growth, simulates campaign outcomes, and recommends how to monetize and scale them.

This pack defines the simple technical foundation needed to begin building Velox using Codex, GitHub, Cloudflare, Supabase, Next.js, and specialized project instructions.

The goal is simple:

Build Velox in a clean, practical, step-by-step way without overcomplicating the first version.

2. Core Velox Build Philosophy

Velox must be built with one guiding principle:

Simple tools first. Powerful tools only when needed.

This avoids the common mistake of trying to build a large enterprise system before the first working version exists.

Velox should first become useful, then intelligent, then scalable.

The Build Logic

1. Start with a clear product vision.
 2. Create a simple working prototype.
 3. Store all work properly in GitHub.
 4. Give Codex strong instructions through AGENTS.md.
 5. Build one feature at a time.
 6. Add data, scoring, and reports.
 7. Add automation and AI memory.
 8. Add Cloudflare infrastructure when scale and control become necessary.
-

3. The Role of Each Tool

3.1 Astra

Astra is the strategic intelligence layer.

Astra helps with:

- Product thinking
- System design
- Feature planning
- Codex commands
- Business model thinking
- Data structure planning
- Creator intelligence logic
- Report structure
- Simplification of complex ideas

Astra turns Cyrus's vision into structured build instructions.

3.2 Codex

Codex is the technical execution partner.

Codex helps with:

- Writing code
- Editing project files
- Creating app pages
- Building components
- Connecting database logic
- Reviewing bugs
- Refactoring code
- Explaining unfamiliar code
- Following project instructions

Codex should not be given vague commands like "Build Velox."

Codex should be given structured commands like:

"Create a Next.js page called Creator Profiles. Use mock data for now. Display creator name, platform, niche, follower count, engagement rate, growth score, monetization readiness, and recommended action."

3.3 GitHub

GitHub is the code memory of Velox.

GitHub stores:

- App files
- Documentation
- Database schema
- Build history
- Codex-generated code
- Issues and future tasks
- AGENTS.md instructions
- Skill files
- Deployment notes

Every serious Velox file should live inside GitHub.

3.4 AGENTS.md

AGENTS.md is the first instruction brain for Codex inside the Velox repository.

It tells Codex:

- What Velox is
- How the app should be built
- What tools are being used
- What design style to follow
- What files not to break
- What coding conventions to follow
- How to report completed work

AGENTS.md should be placed in the root of the Velox GitHub repository.

3.5 Codex Skills

Codex Skills are specialized capability packs.

They can teach Codex how to perform repeated Velox tasks more reliably.

Examples:

- Velox UI Designer Skill
- Velox Database Engineer Skill
- Velox Scoring System Skill
- Velox Report Generator Skill
- Velox Cloudflare Deployment Skill

Skills should come after AGENTS.md, not before. First we define the project brain, then we create specialist skills.

3.6 Next.js / React

Next.js or React will power the user interface.

Velox needs a clean interface for:

- Dashboard overview
- Creator database
- Creator profile pages
- Creator scoring
- Campaign simulation
- Brand matching
- Reports
- Admin controls

Next.js is recommended because it is modern, widely supported, and works well with deployment platforms like Vercel and Cloudflare Pages.

3.7 Supabase

Supabase is recommended for the first Velox backend because it gives us a simple way to manage:

- Database
- Authentication
- Storage
- APIs
- User accounts
- Row-level security

Supabase is easier for the first version than trying to build everything manually.

3.8 Cloudflare

Cloudflare should be used as Velox's infrastructure power layer.

Cloudflare can support:

- Hosting through Cloudflare Pages
- Backend functions through Workers
- SQL-style database through D1
- Object/file storage through R2
- Vector search through Vectorize
- AI inference through Workers AI
- AI monitoring and control through AI Gateway
- Security and performance at the edge

Cloudflare is powerful, but we should introduce it carefully.

For the first prototype, Cloudflare can be used for hosting. Later, it can support AI cost control, vector search, storage, and backend scaling.

4. Recommended Velox Tool Stack

Phase 1: Prototype Stack

This is the simple first version.

Need	Tool
Strategy	Astra
Coding	Codex
Code storage	GitHub
Project instructions	AGENTS.md
Frontend	Next.js / React
Database	Supabase
Authentication	Supabase Auth
Hosting	Vercel or Cloudflare Pages
Design system	Tailwind CSS
Charts	Recharts
Reports	HTML/PDF export later

Phase 2: Intelligence Stack

This is when Velox becomes smarter.

Need	Tool
Creator scoring	Custom scoring logic
AI analysis	OpenAI API
AI usage control	Cloudflare AI Gateway
Creator memory/search	Supabase pgvector or Cloudflare Vectorize
Report generation	AI + template system
File storage	Supabase Storage or Cloudflare R2
Admin workflows	Next.js + Supabase

Phase 3: Scale Stack

This is when Velox becomes infrastructure.

Need	Tool
Global hosting	Cloudflare Pages
Backend APIs	Cloudflare Workers
File/object storage	Cloudflare R2
AI gateway/control	Cloudflare AI Gateway
Vector database	Cloudflare Vectorize
Serverless AI	Workers AI
Advanced security	Cloudflare Access / WAF
Analytics	Cloudflare Analytics + custom dashboard

5. Velox Product Architecture

Velox should be built as a modular system.

Core Modules

1. Creator Database
2. Creator Profile
3. Creator Scoring System
4. Growth Intelligence Dashboard
5. Campaign Simulation Engine
6. Brand Match Engine
7. Monetization Recommendation Engine
8. Report Generator
9. Admin Dashboard
10. Data Collection System

6. First Version Feature List

The first version should not try to do everything.

It should prove the core idea.

Version 1 Must Have

1. Dashboard homepage
2. Creator database table
3. Add creator form
4. Creator profile page
5. Basic scoring system
6. Campaign simulation form
7. Simple recommendation output
8. Report preview page
9. Admin notes section
10. Mock data to demonstrate how Velox works

Version 1 Should Avoid

- Too many AI features at once
- Complex payment systems
- Too many user roles
- Advanced automation before data structure is clear
- Overdesigned dashboards
- Building a mobile app before the web system works

7. Recommended Velox Database Structure

The database should begin simple.

Table: creators

Fields:

- id
- full_name
- creator_name
- platform
- profile_url
- country
- city
- niche
- follower_count
- average_views
- average_likes
- average_comments
- engagement_rate
- growth_rate
- content_frequency
- audience_type

- brand_safety_notes
- monetization_status
- created_at
- updated_at

Table: creator_scores

Fields:

- id
- creator_id
- growth_score
- engagement_score
- consistency_score
- niche_authority_score
- brand_fit_score
- monetization_readiness_score
- overall_velox_score
- score_notes
- created_at

Table: campaigns

Fields:

- id
- brand_name
- campaign_name
- campaign_goal
- target_audience
- budget_range
- preferred_platforms
- required_niche
- start_date
- end_date
- created_at

Table: campaign_simulations

Fields:

- id
- campaign_id
- creator_id
- estimated_reach
- estimated_engagement
- estimated_conversion_potential
- risk_level

- recommendation
- created_at

Table: reports

Fields:

- id
- report_title
- report_type
- creator_id
- campaign_id
- summary
- findings
- recommendations
- created_at

Table: notes

Fields:

- id
- related_creator_id
- note_type
- note_content
- created_by
- created_at

8. Velox Scoring System Foundation

Velox should score creators across six core dimensions.

8.1 Growth Velocity

Measures whether a creator is growing quickly.

Signals:

- Follower growth
- View growth
- Recent traction
- Viral content patterns

8.2 Engagement Quality

Measures whether the audience responds meaningfully.

Signals:

- Comments
- Saves
- Shares
- Like-to-view ratio
- Comment quality

8.3 Consistency

Measures whether the creator posts regularly.

Signals:

- Weekly posting frequency
- Content rhythm
- Gaps in posting
- Format consistency

8.4 Niche Authority

Measures whether the creator owns a clear space.

Signals:

- Topic clarity
- Audience association
- Repeated content themes
- Expertise perception

8.5 Brand Fit

Measures whether the creator can work with brands safely and effectively.

Signals:

- Tone
- Public image
- Content quality
- Audience alignment
- Risk factors

8.6 Monetization Readiness

Measures whether the creator is ready to generate income.

Signals:

- Professionalism
- Audience trust
- Existing brand mentions
- Content packaging
- Conversion potential

Overall Velox Score

The overall Velox Score should combine all six dimensions.

Initial formula:

- Growth Velocity: 20%
- Engagement Quality: 20%
- Consistency: 15%
- Niche Authority: 15%
- Brand Fit: 15%
- Monetization Readiness: 15%

This can later be adjusted based on real campaign performance data.

9. AGENTS.md Draft for Velox

Create a file called AGENTS.md in the root of the Velox repository and paste the following:

```
# Velox AGENTS.md

## Project Identity

Velox is an AI creator intelligence system that discovers emerging creators,
analyzes their growth, simulates campaign outcomes, and recommends how to
monetize and scale them.

The system is being built to support creator discovery, creator scoring,
campaign planning, brand matching, monetization strategy, and intelligence
reporting.

## Build Philosophy

Build simple first. Avoid unnecessary complexity. Prioritize clean structure,
readable code, and practical features that can be tested quickly.

Do not overbuild. Do not add advanced infrastructure unless the current task
```

requires it.

Preferred Stack

- Frontend: Next.js / React
- Styling: Tailwind CSS
- Database: Supabase
- Auth: Supabase Auth
- Hosting: Vercel or Cloudflare Pages
- Future infrastructure: Cloudflare Workers, R2, Vectorize, AI Gateway, Workers AI
- Charts: Recharts
- Code storage: GitHub

Design Style

Velox should feel premium, intelligent, clean, modern, and strategic.

Use:

- Clean dashboard layouts
- Cards
- Tables
- Score badges
- Clear hierarchy
- Minimal clutter
- Professional spacing
- Mobile-responsive layouts

Avoid:

- Overly playful design
- Unnecessary animations
- Confusing navigation
- Overloaded pages

Core Modules

1. Dashboard Overview
2. Creator Database
3. Creator Profile
4. Creator Scoring
5. Campaign Simulation
6. Brand Match
7. Report Generator
8. Admin Notes

Coding Rules

- Keep components reusable.
- Use clear file names.
- Keep logic separated from UI where possible.
- Do not delete existing working code without explaining why.
- When using mock data, clearly label it as mock data.
- Add comments only where they improve understanding.
- Prioritize maintainability over cleverness.

Database Rules

Use simple table structures first. Avoid complex relationships until the feature requires them.

Core tables:

- creators
- creator_scores
- campaigns
- campaign_simulations
- reports
- notes

AI Feature Rules

AI features should support clear product value. Do not add AI for decoration.

AI should help with:

- Creator analysis
- Score explanation
- Campaign recommendation
- Brand matching
- Report writing
- Risk detection

Reporting Format

When completing a task, report:

1. What was changed
2. Which files were edited or created
3. How to test it
4. Any assumptions made
5. Suggested next step

Safety Rules

```
Do not expose API keys.  
Do not hardcode secrets.  
Do not commit sensitive credentials.  
Use environment variables for secrets.
```

Current Product Priority

The current priority is to build Velox Version 1:

1. Dashboard homepage
2. Creator database table
3. Add creator form
4. Creator profile page
5. Basic creator scoring system
6. Campaign simulation form
7. Report preview page

10. First Five Velox Codex Skills

These are the first specialist skills we should create after AGENTS.md is working well.

Skill 1: Velox Product Architect Skill

Purpose:

Helps Codex understand Velox as a product system.

Responsibilities:

- Break big features into smaller tasks
- Keep feature logic aligned with the product vision
- Prevent unnecessary complexity
- Suggest simple feature sequences

Skill 2: Velox UI Designer Skill

Purpose:

Helps Codex create clean, premium dashboard interfaces.

Responsibilities:

- Create dashboard layouts
- Create creator cards
- Create scoring panels

- Create report preview pages
- Maintain visual consistency

Skill 3: Velox Database Engineer Skill

Purpose:

Helps Codex structure the database.

Responsibilities:

- Define Supabase tables
- Create schema files
- Map relationships
- Suggest database improvements
- Protect against messy data structures

Skill 4: Velox Scoring System Skill

Purpose:

Helps Codex build and refine the creator scoring logic.

Responsibilities:

- Implement scoring formulas
- Explain scores
- Add scoring categories
- Build score visualizations
- Prepare future AI-assisted scoring

Skill 5: Velox Report Generator Skill

Purpose:

Helps Codex turn creator and campaign data into intelligence reports.

Responsibilities:

- Generate report layouts
- Create report sections
- Summarize creator intelligence
- Provide recommendations
- Prepare export-ready report views

11. Cloudflare Roadmap for Velox

Cloudflare should not be forced into every part of Velox immediately.

It should be introduced in stages.

Stage 1: Hosting Option

Use Cloudflare Pages as an option for deploying the frontend.

Good for:

- Fast global hosting
- Simple deployment
- Connecting with GitHub

Stage 2: API and Backend Logic

Use Cloudflare Workers for lightweight backend functions.

Good for:

- API routes
- Scoring helpers
- Data processing
- Secure server-side logic

Stage 3: File Storage

Use Cloudflare R2 for storing larger files.

Good for:

- Creator media files
- Report exports
- Screenshots
- CSV uploads
- Dataset archives

Stage 4: AI Monitoring and Control

Use Cloudflare AI Gateway when Velox starts calling AI models frequently.

Good for:

- Tracking AI costs
- Rate limiting

- Caching
- Request retries
- Model fallback
- Observability

Stage 5: Creator Intelligence Memory

Use Cloudflare Vectorize when Velox needs semantic search and memory.

Good for:

- Searching creator profiles
- Comparing creators
- Finding similar creators
- Matching brands to creators
- Retrieval-based AI reports

Stage 6: Serverless AI Tasks

Use Workers AI for selected AI tasks where it makes sense.

Good for:

- Lightweight AI inference
- Embeddings
- Classification
- Edge AI experiments

12. First Velox Build Sequence

This is the recommended order for building.

Step 1: Create the GitHub Repository

Repository name:

```
velox-creator-intelligence
```

Initial files:

- README.md
- AGENTS.md
- /docs
- /app
- /components
- /lib

- /data

Step 2: Add AGENTS.md

Paste the AGENTS.md draft from this pack into the repository.

This gives Codex the first project brain.

Step 3: Ask Codex to Create the App Skeleton

Command:

"Create the initial Next.js app structure for Velox Creator Intelligence. Use Tailwind CSS. Create a clean dashboard layout with sidebar navigation for Dashboard, Creators, Scoring, Campaign Simulation, Reports, and Settings. Use mock data only for now."

Step 4: Build the Dashboard

Command:

"Create the Velox dashboard homepage. Show total creators tracked, average Velox score, fastest growing creator, top niche, campaign simulations, and recent reports. Use clean cards and mock data."

Step 5: Build Creator Database

Command:

"Create a creator database page with a searchable table. Include creator name, platform, niche, country, followers, engagement rate, growth rate, Velox score, and status. Use mock data."

Step 6: Build Creator Profile Page

Command:

"Create a creator profile page that displays creator details, growth metrics, engagement insights, scoring breakdown, brand fit notes, monetization readiness, and recommended next action."

Step 7: Build Basic Scoring Logic

Command:

"Create a reusable scoring function that calculates an overall Velox score based on growth velocity, engagement quality, consistency, niche authority, brand fit, and monetization readiness. Use weighted scoring and return both numeric score and explanation."

Step 8: Build Campaign Simulation Page

Command:

"Create a campaign simulation page where a user can select a brand goal, budget range, target audience, niche, and creator. Generate estimated reach, engagement potential, risk level, and recommendation using mock logic."

Step 9: Build Report Preview Page

Command:

"Create a report preview page that displays a professional Velox creator intelligence report with summary, data insights, scoring breakdown, campaign recommendation, risks, and next steps."

Step 10: Connect Supabase Later

Only after the mock version works should Codex connect Supabase.

Command:

"Prepare the Velox app for Supabase integration. Create database types, Supabase client setup, environment variable placeholders, and schema documentation. Do not remove mock data yet."

13. First Repository Folder Structure

Recommended structure:

```
velox-creator-intelligence/  
  AGENTS.md  
  README.md  
  package.json  
  .env.example  
  /app  
    /dashboard  
    /creators  
    /creators/[id]  
    /scoring  
    /campaigns  
    /reports  
    /settings  
  /components  
    /layout  
    /dashboard  
    /creators
```

```
/scoring
/campaigns
/reports
/ui
/lib
  scoring.ts
  mock-data.ts
  supabase.ts
  utils.ts
/docs
  product-vision.md
  database-schema.md
  scoring-system.md
  deployment-roadmap.md
/skills
  velox-product-architect/
  velox-ui-designer/
  velox-database-engineer/
  velox-scoring-system/
  velox-report-generator/
```

14. README.md Draft

```
# Velox Creator Intelligence
```

Velox is an AI creator intelligence system that discovers emerging creators, analyzes their growth, simulates campaign outcomes, and recommends how to monetize and scale them.

```
## Core Purpose
```

Velox helps brands, agencies, media teams, and creator economy builders identify high-potential creators before the wider market notices them.

```
## Core Modules
```

- Dashboard Overview
- Creator Database
- Creator Profiles
- Creator Scoring
- Campaign Simulation
- Brand Matching
- Report Generation
- Admin Notes

```
## Current Build Phase
```

```
Version 1 prototype using mock data.
```

```
## Tech Stack
```

- Next.js / React
- Tailwind CSS
- Supabase
- GitHub
- Codex
- Future: Cloudflare Pages, Workers, R2, Vectorize, AI Gateway, Workers AI

```
## Build Philosophy
```

```
Simple first. Useful first. Scalable later.
```

15. Immediate Next Action

The next action is not to build every feature.

The next action is to create the Velox repository and install the project brain.

Immediate Action Checklist

1. Create GitHub repository: `velox-creator-intelligence`
2. Add README.md
3. Add AGENTS.md
4. Create `/docs` folder
5. Create `/skills` folder
6. Open repository in Codex or VS Code
7. Ask Codex to create the first Next.js app skeleton
8. Review the result before building the next feature

16. Strategic Positioning

Velox is not only a software project.

Velox is a strategic infrastructure layer for the creator economy.

It can serve:

- Brands
- Agencies
- Funders
- NGOs
- Media houses
- Creator managers
- Influencer campaigns
- Youth training programs
- Film and creative industry ecosystems

The real power of Velox is this:

It helps discover potential before the market has fully priced it.

That is where Velox becomes valuable.

17. Astra Recommendation

The correct next move is to keep the system practical.

Do not overbuild.

Do not chase every tool.

Do not wait until everything is perfect.

Build the first version with mock data, prove the intelligence flow, then connect the database, then add AI, then add Cloudflare infrastructure.

The path is:

**Vision → Structure → Repository → AGENTS.md → Prototype → Database → Scoring → Reports → AI
→ Cloudflare Scale Layer**

This is how Velox becomes real.